



Application development for Linux

Develop with C

Introduction

When developing applications for Linux you have a large selection of programming languages, editors, development environments, libraries and toolchains to choose from. This guide will provide you with instructions for how to get started with application development using the C programming language.

This guide is not a course in C programming or Embedded Linux application development. Instead, it will guide you in setting up the tools that exist for building your first Hello world application.

If you have never worked with Embedded Linux a recommended course is bootlin's Embedded Linux training. The slides are available for download on their site:

<https://bootlin.com/training/embedded-linux/>

Install toolchain

To be able to build an application that will run on Embedded Artists iMX based COM boards a toolchain is needed. The toolchain contains cross compiler, linker, and needed libraries. The steps below show one way to build it, for an alternate way of building a toolchain see Improved Toolchain.

The toolchain can be built in Yocto using the meta-toolchain image.

\$ bitbake meta-toolchain

The build will result in a file located at <build_dir>/tmp/deploy/sdk.

In our example, it is called: fsl-imx-wayland-glibc-x86_64-meta-toolchain-armv8a-imx8mm-smart-toolchain-6.6-scarthgap.sh

We have also a previously build and uploaded install file. It can get by command:

\$ wget http://downloads.influxtechnology.com/Redge/fsl-imx-wayland-glibc-x86_64-meta-toolchain-armv8a-imx8mm-smart-toolchain-6.6-scarthgap.sh

Install the toolchain

Make the installer executable, if needed and start it.

\$./fsl-imx-wayland-glibc-x86_64-meta-toolchain-armv8a-imx8mm-smart-toolchain-6.6-scarthgap.sh

Setting up environment

Before building an application run the command below to setup environment variables. Also you can verify that they has been correctly setup.

```
$ source /opt/fsl-imx-wayland/6.6-scarthgap/environment-setup-armv8a-poky-linux  
$ $CC --version  
aarch64-poky-linux-gcc (GCC) 13.2.0  
...
```

Create simple application

Introduction

To test the toolchain and make sure everything is working a simple Hello world application will be developed.

Source

Create the Hello world application using a text editor. and copy the content of the example below to that file.

```
#include <stdio.h>

int main(int argc, char **argv)
{
    printf("Hello world\n");
    return 0;
}
```

Build

From the terminal where you setup the toolchain environment compile the application. We are giving the application the name hello using the `-o` argument to the compiler.

```
$ $CC -o hello hello.c
```

Create shared object

Introduction

RexGen Smart device uses shared objects, to provide the seed/key mechanism to unlock ECU. You can create own .so, following steps

Source

In `../source/meta-influx/examples` you can find sources of example and calculation tool.

Build

The example.so and seedkey application can build with:

```
$ $CC -fPIC -shared -o hero.so seedkey_hero.c
$ $CC -o seedkey calc_seedkey.c -ldl
```

Copy files to device and use example

```
$ ./seedkey example.so 11223344
22446688
```